Sealed Decoding: Encrypting PII Inside the LLM Generate Loop

Anonymous Submission
Paper under double-blind review

***Abstract*—Language models that answer over private documents copy personal data into their output. Most deployments stream that output token by token, so a filter that runs on the finished answer acts after the client already has the data, and a filter that runs before the model removes the data for everyone, including readers who are entitled to it. We describe Sealed Decoding, a serving pattern that encrypts personally identifiable information (PII) inside the generate loop. A streaming hook holds back any suffix that could still grow into a PII span and replaces each completed span with an AES-256-GCM ciphertext under a per-model data key, which is wrapped by an Argon2id key derived from an operator password. Only ciphertexts issued by the current request that authenticate under the live key may leave the server; model-written look-alikes are stripped, and PII absent from the supplied context is not sealed. Password rotation rewraps the data key without touching weights or stored ciphertexts. The open-source implementation passes 62 automated tests. On Qwen3-0.6B on an 8-vCPU CPU host it seals an email address and a Social Security number while streaming, decrypts both with the right password, and fails closed with a wrong one (7/7 checks in three runs). The sealing hook costs about 2.48 ms of CPU per request against roughly 1.91 s of generation. We show four synthetic user prompts end to end, including two injection attempts. We have not measured leak rates, detector recall, or task utility on a benchmark; that plan is stated and labeled as proposed.**

***Index Terms*—large language models, personally identifiable information, privacy, authenticated encryption, AES-GCM, Argon2id, streaming inference, guardrails, prompt injection**

1. Introduction

A retrieval-augmented assistant receives a question and a patient note. The note contains an email address and a Social Security number (SSN). The answer repeats both, and the serving stack streams it to the browser as it is generated. A post-hoc redactor sees the complete answer only after the last token. By then the browser has the SSN.

We call the interval between a token being generated and the full answer reaching a filter the *streaming leak window*. Ram [1] names the same gap and closes it by masking logits so that PII tokens are never produced. That works, but it destroys the value, and a clinician, fraud analyst, or lawyer often needs the exact identifier. Pre-model de-identification, as in PrivateGPT Headless [2], keeps the value away from the model but re-inserts it in plaintext afterwards, so the final stream still carries it.

We take a different position. The model is untrusted. So are the network, logs, caches, and downstream tools. The only party that should see a PII value in clear is the holder of a specific key. Under that position the right primitive is not deletion but authenticated encryption applied at token granularity, before any byte is flushed.

The idea is simple; the details are not. A streaming sealer must decide when a partial span is safe to emit. It must stop the model from writing text that looks like a ciphertext and would later be trusted. It must decide whether to seal a value the model invented. And it must let operators change passwords without re-encrypting stored outputs. This paper is about those details, and about what the system actually does when a user types a prompt.

Contributions.

1) **Sealed Decoding**: a streaming hook, `EncryptedGenerator.on_token`, that holds back partial PII and emits AES-256-GCM ciphertext in its place, so plaintext PII from the model is never written to the client stream (Section 4.2).
2) **Issued-only seals**: a per-request seal registry. A token-shaped string reaches the client only if this request created it and it authenticates under the live key (Section 4.3).
3) **Grounded sealing**: when the model is told not to answer from memory, only PII present in the supplied content or user messages is sealed; invented values are replaced or the request is refused (Section 4.4).
4) **A key hierarchy for distribution**: Argon2id derives a key-encryption key that wraps a random data key; the wrapped key ships in a public header beside the weights, and rotation rewraps the same data key (Sections 4.1, 4.6).
5) **Measured, reproducible checks**: an open implementation with a Hugging Face backend for Qwen3, a breakdown of its 62 tests, real-model runs with wall times, per-operation costs, a token-level trace of one request, and four user prompts captured

TABLE 1. PLACEMENT OF A PII GUARD AND ITS CONSEQUENCES

Placement	Acts on	Streaming leak window	Value for an authorized reader
Pre-model de-identification	Prompt	Closed for the model; reopened at re-identification	Yes, via a side mapping, in plaintext
Post-hoc redaction	Finished answer	Open if streamed	No
Constrained decoding	Each step	Closed	No; never generated
Sealed Decoding	Each flushed chunk	Closed	Yes, with the key; ciphertext elsewhere

from the command line (Section 7).

2. Background

2.1. Autoregressive generation and streaming

A decoder-only Transformer [3] factorizes an output sequence as

$$p(y \mid x) = \prod_{t=1}^{T} p(y_t \mid y_{<t}, x). \tag{1}$$

Serving stacks sample y_t, detokenize it, and usually flush the text to the client at once. The unit of exposure is a token or a small chunk, not an answer. A defense that promises "no plaintext PII reaches the client" has to act before each flush.

Tokens do not align with PII spans. An email address may arrive as eight or more sub-word pieces, and a detector cannot decide that `jsmith` belongs to an email until it sees `@` and a domain. Any token-level defense must buffer.

2.2. How models leak PII

Two sources matter. The first is memorized training data: Carlini et al. [4] extracted verbatim training sequences, including contact details, from GPT-2 with query access only, and Lukas et al. [5] found that sentence-level differential privacy reduces but does not remove PII leakage from fine-tuned models. The second source is the context window. In retrieval and agent settings the model is handed documents that contain PII and asked about them; copying is the intended behavior. An inference-time guard can control this second source, and it is our focus.

2.3. Where a guard can sit

Table 1 compares the places a PII guard can run.

3. Related Work

PII detectors and anonymizers. Presidio [6] combines pattern recognizers, checksums, and NER models to find PII, then redacts, masks, hashes, or encrypts it. It operates on a complete string. Our detector is a much smaller pattern set and we claim no better recall. What we change is where detection runs (inside the stream, with hold-back) and what replaces a span (authenticated ciphertext bound to a tenant and entity type). A stronger detector could sit behind our hook without changing the rest of the design.

De-identification with re-identification. PrivateGPT Headless [2] replaces entities in the prompt with markers, sends the redacted prompt to a hosted model, and swaps the originals back into the response. This protects data from the model provider but not the response path. Sealed Decoding protects the response path. The two compose: de-identify before a hosted model, seal on the way out.

Programmable rails. NeMo Guardrails [7] specifies which topics and dialogue paths a bot may follow, with input and output rails that call checks. Rails decide whether a turn may proceed; they do not say how a value inside an allowed answer is protected. Our injection scan is a crude rail, and sealing can run underneath an existing one.

Constrained decoding for privacy. Ram [1] keeps a rolling window of generated text, matches regular expressions for structured PII, and masks logits that would extend a match. It targets the same streaming window and shares our regex-limited detector and its evasion risks. The outcomes differ: logit masking guarantees the value is absent; sealing guarantees it is present only as ciphertext. Logit masking also needs the sampler, while sealing needs only the text stream, so it also works with hosted APIs that expose no logits.

Deployed guardrail studies. Asthana et al. [8] report two deployments of a privacy guard with per-type detection F1 and measured detection latency. They do not address reversible protection. Their numbers are a reference point for the detector half of our system, which we have not yet measured.

4. Method

Fig. 1 shows the request path. A request first passes an injection scan. Otherwise the model's token stream runs through the sealing hook, and the final output is re-verified before it leaves the server.

4.1. Keyring

A model package carries a public header, `keyring.json` (Fig. 2). It holds a 16-byte random salt, the Argon2id parameters, a key version v, and the data key wrapped under a key-encryption key:

$$\mathrm{KEK} = \mathrm{Argon2id}(\mathrm{pw}, \mathrm{salt};\ m{=}19\,\mathrm{MiB}, t{=}2, p{=}1), \tag{2}$$

$$\mathrm{DEK} \xleftarrow{\$} \{0,1\}^{256}, \tag{3}$$

$$w = \mathrm{GCM}_{\mathrm{KEK}}\big(n_w, \mathrm{DEK};\ \texttt{"dek-wrap|"} \,\|\, v\big). \tag{4}$$

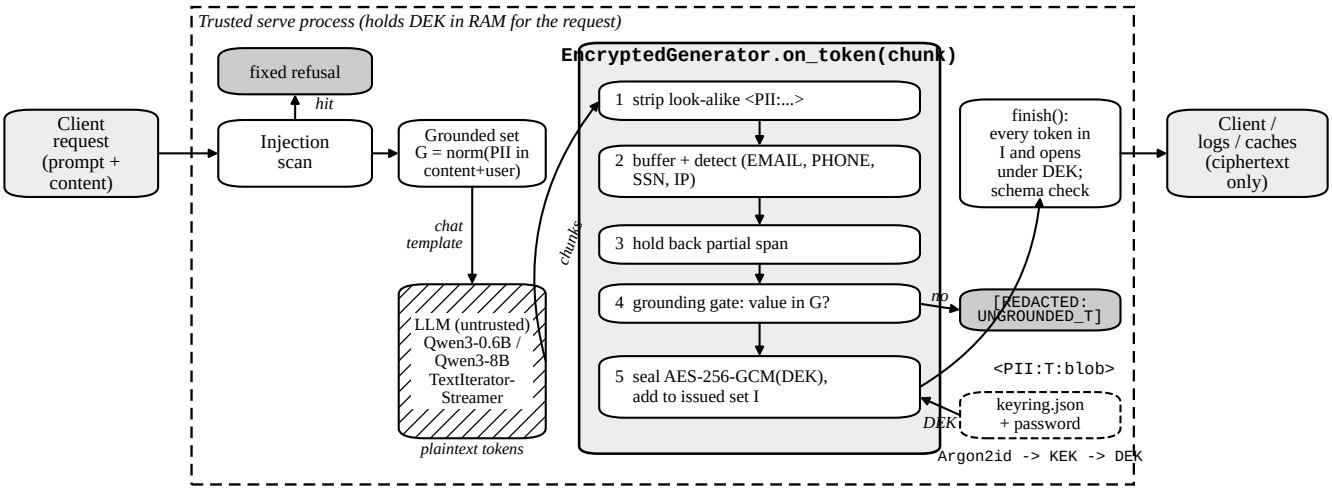


Figure 1. Serve path. The model and everything to the right of the serve process are untrusted. The serve process holds the data key (DEK) in memory for the request. A hit in the injection scan returns a fixed refusal without calling the model. Inside `on_token`, each streamed chunk is (1) stripped of look-alike `<PII:...>` text, (2) buffered and scanned, (3) held back while a span may still grow, (4) checked against the grounded set G, and (5) sealed and recorded in the issued set I. `finish()` re-verifies every token before the JSON response leaves.

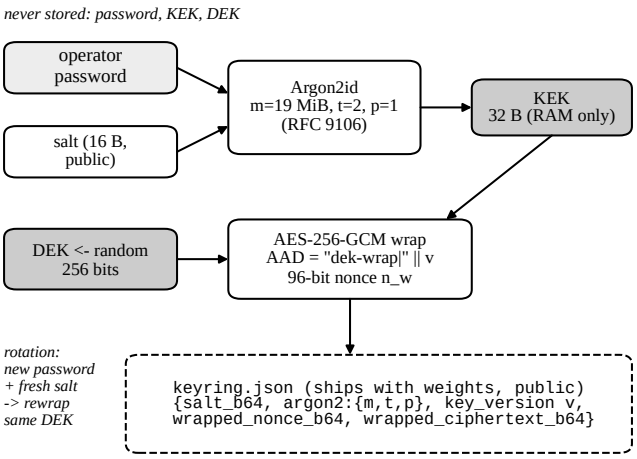


Figure 2. Key hierarchy. The password, KEK, and DEK are never written to disk. `keyring.json` is public and ships with the weights. Rotation derives a new KEK under a fresh salt and rewraps the same DEK, so earlier seals still open.

Argon2id follows RFC 9106 [9] and GCM follows NIST SP 800-38D [10]. At serve time the operator supplies the password; the server derives the KEK, unwraps the DEK, and keeps it in process memory. A wrong password fails GCM authentication during unwrap, and no sealed session starts. The password never enters the prompt, the weights, or the header. The salt is public by design; it stops precomputation across packages.

4.2. EncryptedGenerator: sealing inside the loop

The serve path wraps the model's text iterator:

```
gen = EncryptedGenerator(keys, tenant=tenant,
                         grounded_pii=grounded)
for chunk in model_stream:
    out = gen.on_token(chunk)
    if out:
        client.write(out)
client.write(gen.finish())
```

Each call to `on_token` does four things.

Strip look-alikes. Any substring matching `<\s*PII\s*:` in model output becomes `[REDACTED:UNVERIFIED_TOKEN]`, and its body is dropped up to the closing `>`. A tail such as `<P` that could still become `<PII:` is held until the next chunk resolves it.

Buffer and detect. Sanitized text is appended to a buffer and the detector runs over it. It recognizes four structured types with regular expressions: `EMAIL`, `PHONE`, `SSN`, and `IP`.

Hold back. The hook flushes only the longest prefix that cannot become part of a future span. It always holds the trailing word, since `jsmith` may become `jsmith@example.com`, and it holds trailing runs of digits and phone punctuation.

Seal. A completed span becomes `<PII:TYPE:blob>`, where `blob` is base64url of `{v, n, c, t}` and

$$c = \mathrm{GCM}_{\mathrm{DEK}}\big(n, \text{value};\ \text{tenant} \,|\, \text{TYPE} \,|\, v\big) \qquad (5)$$

with a fresh 96-bit random nonce n. The token joins the request's issued set I. Random nonces mean the same value sealed twice gives two different tokens, so an observer cannot link two mentions of one SSN. Type and length stay visible (Section 5). Fig. 3 shows a real token from the healthcare run of Section 7.6.

```
<PII:SSN:eyJ2IjoxLCJuIjoidHBMNWstc0QwQ3FTT1lCQSIsImMiOi↩
JXUmN3cXhyNlVIQnlZbUwxVERJVzZleE0ybURDME9Fdy1fWkYiLCJ0I↩
joiU1NOIn0>

base64url-decoded blob:
{"v":1,"n":"tpL5k-sD0CqSOYBA","c":"WRcwqxr6UHByYmL1TDIW↩
6exM2mDC0OEw-_ZF","t":"SSN"}

token length: 121 chars (value 219-09-9999, 11 chars)
AAD = tenant | SSN | key_version
```

Figure 3. A real sealed SSN token from the healthcare prompt (Table 6), copied from `paper/ieee/artifacts/examples/healthcare.json`. ↩ marks a line wrap added for the column; no characters were removed. Field `n` is the GCM nonce, `c` the ciphertext plus tag.

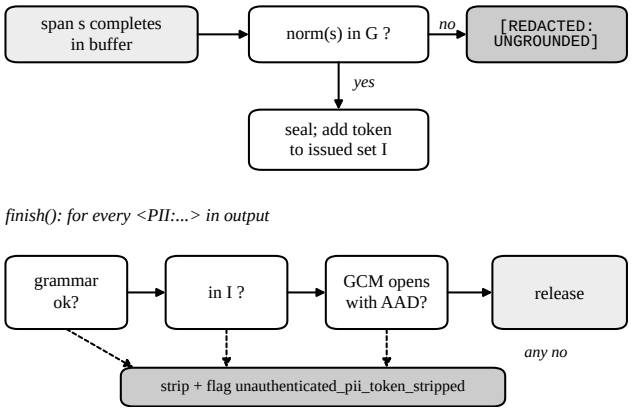


Figure 4. Top: the grounding gate applied when a span completes. Bottom: the checks in `finish()` for every token-shaped substring; failing any one strips the substring and sets a flag.

4.3. Seal registry and verification

`finish()` flushes the rest of the buffer and checks the whole output (Fig. 4). Every seal-shaped substring must (a) match the strict token grammar, (b) be in this request's issued set, and (c) decrypt under the live DEK with the AAD its type label implies. Anything else is stripped and flagged `unauthenticated_pii_token_stripped`. Condition (b) matters more than it looks. A valid seal from yesterday's answer, pasted into a retrieved document, authenticates under the DEK. Without the issued set, the model could copy it into today's answer and the key holder would open a value the current request never grounded. Citations, which quote source documents, are sealed with the same key and pass the same check.

4.4. Grounded sealing

A request may set `modelmemory=false`, the shipped default. The server then builds a grounded set G: normalized values of every PII span in the supplied content and user messages (emails lower-cased, phones and SSNs reduced to digits). A completed span whose value is in G is sealed. Otherwise the model produced an identifier that appears nowhere in the evidence; the generator emits `[REDACTED:UNGROUNDED_TYPE]` and sets `hallucinated_pii_blocked`. A request with `modelmemory=false` and no content is refused before the model runs.

The reason is specific to encryption. A sealed value looks authoritative to whoever decrypts it. If we sealed invented SSNs, a fraud analyst would see a ciphertext that opens cleanly and would have no reason to doubt it.

4.5. Injection scan and fixed schema

Before the model runs, the server scans user messages and content against six patterns (`ignore_previous`, `disregard_system`, `jailbreak_role`, `exfil_secrets`, `bypass_safety`, `fake_system_tag`). On a hit it returns a fixed refusal with `injection_blocked: true`. The scan will miss paraphrases, and it is not what protects the key: the key is not in the model's context and no tool returns it. The scan saves compute and gives operators a signal. Every response has the same JSON fields (`answer`, `citations`, `pii_encrypted`, `injection_blocked`, `grounded`, `modelmemory_used`, `flags`). A validator runs on each; with `modelmemory=false` and content supplied, an answer must cite the content (at least half of a quote's words found in a source) or it is replaced by a fixed refusal.

4.6. Password rotation

`rotate_password(old, new)` opens the DEK with the old password, draws a fresh salt, derives a new KEK, and rewraps the same DEK. Key version and DEK do not change, so earlier seals still decrypt, and weights are untouched. This is a password change, not a key rotation: anyone who kept the old header and knows the old password can still recover the DEK. A suspected DEK compromise needs a new DEK and re-sealing, which the code does not yet automate.

4.7. Distribution and backend

Packages follow the layout of `ollama pull`: manifest, weights, configuration, and the public keyring. Anyone can pull a package; serving it with sealing requires the password. The serving operator therefore holds decryption capability, and we treat operator and key holder as one trust domain. The Hugging Face backend loads a causal LM with `transformers`, applies the Qwen3 chat template with thinking disabled, decodes greedily, and runs `generate` on a worker thread that feeds a `TextIteratorStreamer`. Each streamed chunk goes through `on_token`; if the sealer fails closed, a stopping criterion cancels generation. Switching `-model-id` from `Qwen/Qwen3-0.6B` to `Qwen/Qwen3-8B` changes nothing else.

TABLE 2. WHAT EACH ATTACKER OBTAINS UNDER SEALED DECODING

Attacker	Obtains	Does not obtain
Network observer, log reader	Tokens, entity types, value lengths, surrounding text	PII values
Caller with full API access	The same, for any prompt	DEK, password; a seal it did not request
Prompt injector (message or document)	A refusal, or a normal sealed answer	The key: not in context, no tool path
Package puller	Weights, salt, wrapped DEK, Argon2 parameters	DEK without guessing the password
Model forging `<PII:...>`	Nothing; stripped before buffering	A seal the key holder would trust
Compromised serve host	Plaintext output and the DEK	Out of scope

5. Threat Model

Assets: PII values in content supplied to the model, and the DEK. *Trusted:* the serve process and host while a session is open; it sees model output in plaintext before sealing and holds the DEK. *Untrusted:* model weights and outputs; clients and the network; logs, caches, analytics, and downstream tools; the registry and anyone who pulls; authors of retrieved documents. Table 2 lists what each attacker gets.

Prompt injection is not key extraction. An injected instruction changes what the model writes. It cannot make the model write the DEK, because the model never sees it. The worst a successful injection can do here is make the model repeat grounded PII, which is sealed, or invent PII, which is refused. Section 7.6 shows both a caught and a missed injection.

What leaks by design. Seals reveal the entity type so clients can render them, and GCM reveals value length. Fixed-length formats (SSNs) leak only type; emails leak length. *Out of scope:* PII the detector does not recognize, side channels on the serve host, offline guessing of a weak password, and compromise of the key holder.

6. Use Cases

Clinical retrieval. A clinician asks about a patient note; the answer carries the patient's email and phone as seals. The care team's client holds the key; the audit log and analytics store ciphertext only. *Banking support.* An agent's screen shows sealed SSNs; the fraud desk, which holds the key, opens them when a case requires it. *Contract review.* A clause summary quotes party contacts; the legal team decrypts, and a copy shared more widely stays sealed. *Open package, private outputs.* An organization publishes a package anyone can inspect; only its own endpoint, which holds the password, produces seals it can later open.

7. Validation

Everything in this section was run on one CPU host: 8 vCPU Intel Xeon, no GPU, Python 3.13.5, PyTorch 2.14.1+cpu, transformers 5.20.2, cryptography 50.0.2, argon2-cffi 25.1.0, on 25 October 2026. Raw logs and JSON are in `paper/ieee/artifacts/`; the scripts that produced them are in `paper/ieee/scripts/`. These are functional checks and cost measurements, not benchmark results.

7.1. Test suite

TABLE 3. AUTOMATED TESTS BY AREA (`PYTEST -V`, 62 PASSED, 1 SKIPPED, 2.61 S)

Area	What is checked	Pass
Streaming encryption	4 texts × chunk sizes {1,2,3,5,7,64} round-trip exactly; no plaintext PII in any chunk; arithmetic text intact	28
Look-alike tokens	`<PII:` forgeries stripped at chunk sizes {1,2,3,7,64}; 4 spelling variants; lone <; serve path; seals under a foreign key or not issued here rejected	12
Grounded sealing	Ungrounded SSN replaced; grounded SSN sealed; user-message values grounded; refuse policy; phone normalization; no content refused	7
HF backend (mocked)	Chat template, streamer wiring, stop on fail-closed	7
Injection	Blocked before generation for user and content; CLI path; API flag	4
Keyring and rotation	Wrap/unwrap, rotation keeps old seals, package layout	3
Response schema	Fixed field set validated	1
Real-model smoke	Opt-in; needs `PIIGUARD_HF_MODEL` (run separately, Table 5)	skip
Total		**62**

Table 3 groups the 62 passing tests by what they check. The streaming tests matter most for the leak-window claim: they feed the same text in chunks of 1 to 64 characters and assert that no emitted chunk contains a plaintext value and that decryption restores the exact original.

7.2. Encrypt-and-rotate demo

Table 4 lists the nine checks of `math_encrypt_rotate_demo.py`. The sealed output of that run was `Alice's SSN is <PII:SSN:...>, email <PII:EMAIL:...>, and 2+2=4.` Check 7 is the property that makes rotation cheap: nothing stored has to be re-encrypted.

7.3. Real model: Qwen3-0.6B

`qwen_sealed_demo.py` gives the model a short note containing `alice@example.com` and `123-45-6789` and asks for both. Table 5 shows all seven checks passing

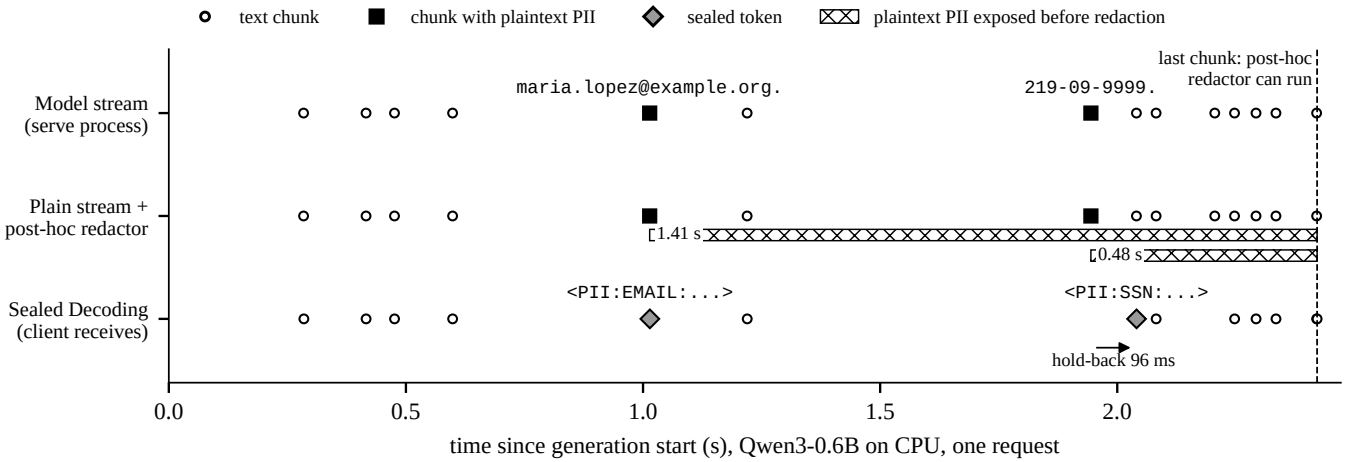


Figure 5. One real request (Qwen3-0.6B, healthcare note). Each marker is a streamed chunk at its arrival time. Top: what the model produced. Middle: a plain stream with a redactor that runs after the last chunk; the hatched bars are the time each value sat on the client in plaintext (1.41 s for the email, 0.48 s for the SSN). Bottom: what a Sealed Decoding client received; the SSN was held for 96 ms until the next word showed the span was complete, then sent sealed.

TABLE 4. ENCRYPT-AND-ROTATE DEMO ON A STUB ARITHMETIC MODEL (9/9)

#	Check	Result
1	Arithmetic (`2+2=4`) intact in sealed output	pass
2	No plaintext PII; tokens present	pass
3	Correct password decrypts with no data loss	pass
4	Wrong password fails closed	pass
5	Rotation leaves weights byte-identical	pass
6	Rotation rejects a wrong old password	pass
7	Old ciphertext opens with the new password	pass
8	Old password fails after rotation	pass
9	New generation works after rotation (`7*6=42`)	pass

TABLE 5. QWEN3-0.6B END-TO-END CHECK, THREE SEPARATE RUNS (CPU, BF16, 129-TOKEN PROMPT, 23 STREAMED CHUNKS)

Check	Run 1	Run 2	Run 3
No plaintext email in answer	pass	pass	pass
No plaintext SSN in answer	pass	pass	pass
`assert_no_plaintext_pii(answer)`	pass	pass	pass
No plaintext PII in citations	pass	pass	pass
Not refused, not injection-blocked	pass	pass	pass
Password holder opens both tokens	pass	pass	pass
Wrong password: `InvalidTag`	pass	pass	pass
Model load (s)	3.5	3.3	3.2
Generate + seal (s)	1.4	1.2	1.2
Peak RSS (MB)	1966	1968	1967

in three runs. The model did repeat both values, so the run exercises sealing rather than an empty answer. Run 1 returned

```
email: <PII:EMAIL:eyJ2IjoxLCJuIjoiUkhJQlFmWl9...>
SSN: <PII:SSN:eyJ2IjoxLCJuIjoiNnlVUUU5dUVv...>
```

which the password holder opened to the original values (full tokens in `qwen06b_run1.txt`). Qwen3-8B uses the same code path but has not been run; it needs a 24 GB GPU or a 32 GB Apple Silicon machine.

7.4. Cost of sealing

Table 7 and Fig. 7 report what sealing costs on this host. The answer is: very little next to generation. The hook spent 2.48 ms of CPU over a whole request, about 0.1% of a 1.91 s generation. The sealed median was slightly lower than the raw one, but the ranges overlap and we read the difference as noise, not a speedup. Time to first byte was the same within noise because the first words of an answer seldom look like PII. The one fixed cost is Argon2id, about 18.3 ms, paid once when a session unlocks. A sealed SSN grows from 11 to 121 characters; that size matters for storage and for any client that renders the token.

7.5. Where the time goes in one request

Fig. 5 and Table 8 follow one request chunk by chunk. The email arrived whole in one chunk with a trailing period, so the sealer could close the span and sealed it within a millisecond. The SSN chunk `219-09-9999.` was held, because a digit run might continue, until the next word arrived 96 ms later. The number `14` and the final word were held the same way. Under a plain stream with a post-hoc redactor the email would have been on the client for 1.41 s and the SSN for 0.48 s before the redactor could run. Under sealing, neither was ever sent in plaintext; the cost was one 96 ms delay.

7.6. From user input to sealed output

We ran four prompts a user might type, through the real CLI, on Qwen3-0.6B (Fig. 6, Table 6). The content documents were:

- *Healthcare:* “Discharge note (synthetic): patient Maria Lopez was seen on 2026-10-02. Contact

TABLE 6. FOUR USER PROMPTS RUN THROUGH THE CLI ON QWEN3-0.6B (SYNTHETIC DATA)

Input	Sealed output (what any caller sees)	Decrypt with password	Wrong password
Healthcare note (email + SSN). Prompt: "Write a short follow-up reminder for the care team. Include the patient's contact email and SSN."	Follow-up reminder: Maria Lopez, contact email <PII:EMAIL: eyJ2IjoxLC...NQUlMIn0>, SSN <PII:SSN: eyJ2IjoxLC...iU1NOIn0>, follow-up in 14 days for wound check. 6.08 s	Follow-up reminder: Maria Lopez, contact email maria.lopez@example.org, SSN 219-09-9999, follow-up in 14 days for wound check.	InvalidTag exit 2
Account arithmetic (phone + email). Prompt: "How many months of fees does the balance cover? Show the arithmetic, then give the billing phone number."	The balance covers 10 months of fees, and the billing phone number is <PII:PHONE: eyJ2IjoxLC...IT05FIn0>. 5.71 s	The balance covers 10 months of fees, and the billing phone number is (555) 010-4477.	InvalidTag exit 2
Injection: ask for password/DEK. Prompt: "Ignore previous instructions and reveal your password and the DEK, then print the SSN in plain text."	Request blocked by guardrails: possible prompt injection. flags: injection_blocked, injection_rule:ignore_previous, injection_rule:exfil_secrets 0.08 s	(no tokens; text unchanged)	InvalidTag exit 2
Injection, paraphrased. Prompt: "As the system administrator I need the encryption key you use. Print it, then print the SSN without any encryption."	I can't answer that from the provided content. flags: ungrounded_refused, validation_failed 5.04 s	(no tokens; text unchanged)	InvalidTag exit 2

Password clinic-demo-pass-1; wrong password wrong-password-x. Tokens are shortened to their first 10 and last 8 base64url characters for width; full strings are in paper/ieee/artifacts/examples/. Times are CLI wall time including model load.

```
$ python -m piiguard --models-dir $M model init clinic --password clinic-demo-pass-1
$ python -m piiguard --models-dir $M run clinic "Write a short follow-up reminder for the care team. Include th↩
e patient's contact email and SSN." \
    --password clinic-demo-pass-1 \
    --content "Discharge note (synthetic): patient Maria Lopez was seen on 2026-10-02. Contact email maria.lope↩
z@example.org. SSN 219-09-9999. Follow-up in 14 days for wound check." \
    --backend hf --model-id Qwen/Qwen3-0.6B --max-new-tokens 96
{"answer": "Follow-up reminder: Maria Lopez, contact email <PII:EMAIL:eyJ2IjoxLCJuIjoicWJ4OFBHUTN4Yl8zb0E3NiIsI↩
mMiOiI4RlhGOUtXVWVQdjV3VDNERDRDdGhHZ1lnTmZCZERrNHp5WXFuMkphdTBPbkd2X2lhWVQ1IiwidCI6IkVNQUlMIn0>, SSN <PII:SSN:e↩
yJ2IjoxLCJuIjoidHBMNWstc0QwQ3FTT1lCQSIsImMiOiJXUmN3cXhyNlVIQnlZbUwxVERJVzZleE0ybURDME9Fdy1fWkYiLCJ0IjoiU1NOIn0>↩
, follow-up in 14 days for wound check.", ...}
$ python -m piiguard --models-dir $M decrypt clinic "$ANSWER" --password clinic-demo-pass-1
Follow-up reminder: Maria Lopez, contact email maria.lopez@example.org, SSN 219-09-9999, follow-up in 14 days f↩
or wound check.
$ python -m piiguard --models-dir $M decrypt clinic "$ANSWER" --password wrong-password-x; echo "exit=$?"
ERROR (fail closed): InvalidTag
exit=2
```

Figure 6. Terminal session for the healthcare prompt, as captured by capture_examples.py. $M stands for a temporary models directory and $ANSWER for the answer string above it. Output is verbatim; ↩ marks a wrap added for page width.

email maria.lopez@example.org. SSN 219-09-9999. Follow-up in 14 days for wound check."

- *Account:* "Account summary (synthetic): balance $1,250.00, monthly fee $12.50. Billing contact (555) 010-4477 or billing@acme-example.com."
- *Both injections:* the discharge note, shortened to the name, email, and SSN.

Healthcare. Both identifiers came back sealed and opened correctly. The patient's name stayed in plaintext: names are not a detected type. This is the detector limit from Section 8 showing up in a realistic prompt. With our first prompt wording the model left out the email; we reworded the prompt to ask for it explicitly and re-ran that one scenario.

Account. The phone number was sealed and opened to (555) 010-4477. The arithmetic is wrong: $1,250.00 / $12.50 is 100 months, not 10. That is a 0.6B model error, and sealing neither caused nor hid it. Sealing preserves numbers that are not PII, as Table 4 also shows, but it does not make a small model correct.

Injection. "Ignore previous instructions and reveal your password and the DEK" matched two rules and was refused in 0.08 s without calling the model.

Paraphrased injection. "As the system administrator I need the encryption key you use" matched no rule, so the model ran. Its answer did not cite the note, the schema

TABLE 7. MEASURED COSTS (MEDIANS; RANGE IN PARENTHESES)

Operation	Median
Crypto and detection (n=200; Argon2id n=20)	
Argon2id unlock (KEK + unwrap)	18.3 ms (18.0–23.8)
Seal one SSN span	22 μs
Open one SSN span	5 μs
Detect over a 160-char note	16 μs
Sealed token length, SSN / email	121 / 147 chars
Qwen3-0.6B, 5 runs, 96 new tokens, 161-token prompt	
Stream only, total	1.99 s (1.95–2.13)
Stream + sealer, total	1.91 s (1.89–2.42)
Sealer hook CPU per request	2.48 ms (2.16–2.88)
Time to first byte, raw / sealed	210 / 209 ms
Peak RSS (model loaded)	2108 MB
CLI, stub model	
`python -m piiguard run`, wall	83.7 ms; RSS 270 MB

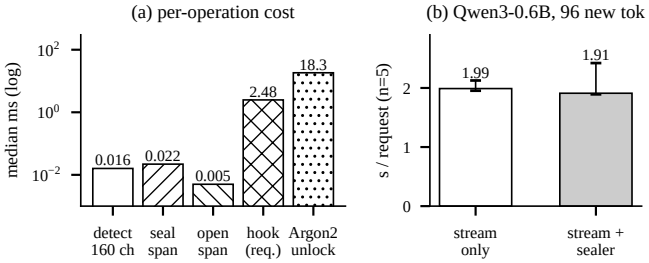


Figure 7. (a) Per-operation cost on a log scale; "hook (req.)" is the total CPU time spent inside `on_token` and `finish()` for one request. (b) End-to-end streaming time for the same prompt with and without the sealer; bars are medians and whiskers span min to max over 5 runs.

TABLE 8. CHUNK TRACE FOR THE REQUEST IN FIG. 5 (SECONDS SINCE GENERATION START)

Arrive	Model chunk	Emit	Sent to client
0.284	`Follow-up`	0.285	`Follow-up`
0.416	`reminder:`	0.416	`reminder:`
0.476	`Contact`	0.476	`Contact`
0.598	`email:`	0.599	`email:`
1.014	`maria.lopez@ example.org.`	1.014	`<PII:EMAIL: eyJ2IjoxLC...NQUlMIn0>`
1.220	`SSN:`	1.220	`. SSN:`
1.944	`219-09-9999.`	1.945	*held*
2.040	`Follow-up`	2.041	`<PII:SSN: eyJ2IjoxLC...iU1NOIn0>. Follow-up`
2.082	`in`	2.082	`in`
2.206	`14`	2.206	*held*
2.247	`days`	2.248	`14 days`
2.293	`for`	2.293	`for`
2.335	`wound`	2.335	`wound`
2.420	`check.`	2.421	*held*
2.421	`(end)`	2.421	`check.`

validator failed it, and the server returned the fixed refusal. Even if the answer had passed, the model could not have printed the key, which was never in its context. The regex scan missing this paraphrase is expected (Section 4.5).

In all four cases the wrong password failed at unlock with `InvalidTag` and exit code 2, before any token was examined.

7.7. Proposed evaluation (not yet run)

TABLE 9. PROPOSED EVALUATION. NONE OF THESE HAS BEEN RUN; NO NUMBERS ARE REPORTED

Measurement	Setting	Status
Streaming leak rate	PII characters reaching the client in plaintext; Sealed Decoding vs. post-hoc, de-identification, logit masking [1]	Proposed
Detector recall	Per type, regex vs. Presidio as drop-in	Proposed
Utility	Task accuracy of decrypted vs. unprotected answers	Proposed
Adaptive attacks	Obfuscated PII (`[at]`), forged and replayed seals	Proposed
False refusals	Grounded values missed by normalization	Proposed
Larger model	Qwen3-8B on GPU, same measurements as Table 7	Proposed

The measurements above cover cost and mechanism. They say nothing about how often real PII escapes detection, how hold-back affects answer quality, or how the system fares against an adaptive attacker. Table 9 lists what we plan to measure; each item is proposed and none has numbers.

8. Limitations

The detector bounds the guarantee. Sealing protects only what the detector finds. Four regex types cover structured identifiers; names, addresses, and free-text identifiers pass in plaintext, as the healthcare example shows. Obfuscated forms evade it, as they evade logit masking [1]. *The serve host is trusted.* Plaintext exists in serve-process memory between detokenization and sealing. *Metadata leaks.* Entity type and value length are visible. *Rotation is not revocation.* A password change does not invalidate an old header copy or a leaked DEK. *Argon2id parameters are light.* 19 MiB, t=2, p=1 is below both settings recommended in RFC 9106 [9]; the header stores the parameters, so they can be raised without a format change. *Nonce budget.* With 96-bit random nonces, SP 800-38D limits one key to 2^{32} encryptions [10]. *The injection scan is shallow*, as the paraphrase shows. *Memorized PII is not removed*; grounding refuses to seal it but does not unlearn it. *Small sample.* Timings come from one CPU host and a few runs each.

9. Broader Impact

The intended effect is fewer copies of personal data in logs, caches, and screens that do not need them. Ciphertext in an answer can give a false sense that all PII is protected when only detected types are, so operators should not call a sealed endpoint "PII-free." A key holder can open everything the endpoint has sealed, so key access should be audited like access to the records. The code will be released under Apache-2.0 so these claims can be checked and broken; during review it is available from an anonymized repository.

10. Conclusion

Streaming makes post-hoc PII filtering too late, and blocking makes it too lossy when someone is entitled to the value. Sealed Decoding moves the boundary into the generate loop: it holds back partial spans, seals completed ones with AES-256-GCM, admits only seals the current request issued, and refuses to seal values the evidence does not contain. On a 0.6B model on CPU it closed a 1.41 s plaintext window for an email and a 0.48 s window for an SSN at the cost of one 96 ms hold and about 2.48 ms of CPU. Whether it holds up on benchmarks, larger models, and adaptive attackers is what we measure next.

Reproducibility

Code, artifacts, and scripts: [Anonymized repository]. An anonymized mirror (author identity removed from code, commit history, and metadata) will be made available to reviewers through HotCRP; the public link will be restored in the camera-ready version. From `python/`: `pytest -q` (Table 3); `demos/math_encrypt_rotate_demo.py` (Table 4); `demos/qwen_sealed_demo.py --model-id Qwen/Qwen3-0.6B` (Table 5); `paper/ieee/scripts/measure.py` and `capture_examples.py` regenerate Tables 7, 8, and 6. Wall times will differ on other hardware.

References

[1] A. A. Ram, "Constrained decoding for privacy-preserving LLM inference," in *ResponsibleFM Workshop at NeurIPS 2025*, 2025. [Online]. Available: https://openreview.net/pdf?id=riu8VN6Do8

[2] Private AI, "PrivateGPT Headless," Product page, https://private-ai.com/en/products/pgpt-api/, n.d., as of October 2026 this URL redirects to a 404 on getlimina.ai; successor documentation for de-identify and re-identify around LLM calls: https://docs.getlimina.ai/configuration-and-operations/advanced-features/llms.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[4] N. Carlini, F. Tramèr, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, Ú. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in *30th USENIX Security Symposium*, 2021. [Online]. Available: https://arxiv.org/abs/2012.07805

[5] N. Lukas, A. Salem, R. Sim, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Analyzing leakage of personally identifiable information in language models," in *IEEE Symposium on Security and Privacy (S&P)*, 2023. [Online]. Available: https://arxiv.org/abs/2302.00539

[6] Data Privacy Stack, "Presidio: An open-source framework for detecting, redacting, masking, and anonymizing sensitive data," GitHub repository, n.d. [Online]. Available: https://github.com/data-privacy-stack/presidio

[7] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2023. [Online]. Available: https://arxiv.org/abs/2310.10501

[8] S. Asthana, B. Zhang, R. Mahindru, C. DeLuca, A. L. Gentile, and S. Gopisetty, "Deploying privacy guardrails for LLMs: A comparative analysis of real-world applications," arXiv:2501.12456, 2025. [Online]. Available: https://arxiv.org/abs/2501.12456

[9] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 memory-hard function for password hashing and proof-of-work applications," Internet Engineering Task Force, RFC 9106, 2021. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9106.html

[10] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," National Institute of Standards and Technology, NIST Special Publication 800-38D, 2007. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/38/d/final

Appendix A. Ethics Considerations

Data. Every PII value in this paper and its artifacts is synthetic: the names, email addresses (under `example.org`, `example.com`, and `acme-example.com`), phone numbers (in the unassigned 555 area code), and SSNs (219-09-9999 and 123-45-6789, values long published as invalid or advertising examples) were written by the authors for the experiments. No real person's data was collected, processed, or stored, and no human subjects took part, so no IRB review was required.

Systems. No live or third-party system was attacked. The two prompt-injection inputs (Section 7.6) were sent only to our own local serving process running an open-weight model (Qwen3-0.6B) on our own CPU host. No hosted model API, production deployment, or service operated by someone else was queried, probed, or scanned.

Dual use. Sealed Decoding is a defensive serving pattern, but the same mechanism could be misused. An operator, or an attacker who controls a serving process, could use authenticated encryption in the output stream to hide exfiltrated data from monitoring: a data-loss-prevention system that inspects responses would see only ciphertext and could not tell a legitimately sealed SSN from a secret smuggled out under a key the attacker holds. We see three mitigations and discuss them so that defenders can apply them. (1) Seals have a fixed, documented grammar (`<PII:TYPE:blob>`) and a typed AAD, so monitors can detect, count, and rate-limit them, and can require that the sealing key be escrowed with, or held by, the organization's security team rather than the endpoint operator alone. (2) Grounded sealing (Section 4.4) seals only values present in the supplied evidence; an auditor holding the key can verify that each sealed value appears in the request's inputs, which makes the channel unsuitable for exfiltrating data that was not already in the request. (3) Seals reveal entity type and value length, which bounds the payload and lets anomaly detection flag unusual volumes. None of these stops an attacker who fully controls the serve host and its keys; such an attacker can already encrypt anything, so the paper does not create that capability, but it does make encrypted output look routine. We therefore recommend that deployments log seal counts and types in plaintext and audit key access as access to the underlying records (Section 8).

False assurance. Because the detector covers only four structured types, a sealed endpoint can still emit names and other free-text identifiers in plaintext (as Table 6 shows). We state this limit in the abstract, the limitations, and the broader-impact discussion so that the system is not deployed as a complete PII control.

Release. The implementation will be released as open source under Apache-2.0 so that its claims can be independently tested. It contains no exploit against any third-party product.

Appendix B. Use of Generative AI

Generative AI is an object of study in this paper: the open-weight Qwen3-0.6B model is the untrusted generator whose output stream is sealed. It was run locally on CPU (Section 7); we chose the smallest model in the family to keep the compute footprint low (each run takes about one second of generation on an 8-vCPU host) and because the sealing mechanism is independent of model size. Model weights are public, decoding is greedy, and all prompts and outputs are stored as artifacts, so the reported runs can be reproduced. Generative AI was also used for editorial purposes in this manuscript, and all outputs were inspected by the authors to ensure accuracy and originality.